# Контест 14 · Финал сезона

Неделя 28 · весь год: рейтинги и турниры, протоколы (HTTP, WebSocket, initData), графы и миграции, релиз и публикация репозитория; в проектной части — документация, release candidate и сценарий демо. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

> Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

## A. Игры

**A1.** `elo(ra, rb, score, k=32)` — рейтинг Эло: ожидание A `= 1 / (1 + 10^((rb − ra) / 400))`, `score` — 1, 0.5 или 0 для A. Изменение `k · (score − ожидание)` прибавить A и вычесть у B, округлить до целых.

```
Ввод:  elo(1500, 1500, 1), elo(1600, 1400, 0), elo(1200, 1400, 0.5)
Вывод: (1516, 1484), (1576, 1424), (1208, 1392)
```

**A2.** `leaderboard(scores)` — места «1224»: `(место, имя, очки)` по убыванию очков, при равенстве — по имени. Равные очки — равное место, следующее место пропускается.

```
Ввод:  leaderboard({"ann": 10, "bob": 12, "cid": 10, "dan": 7, "eve": 12})
Вывод: [(1, "bob", 12), (1, "eve", 12), (3, "ann", 10), (3, "cid", 10), (5, "dan", 7)]
```

**A3.** `validate_move(msg)` — проверка хода из WebSocket: `{"type": "move", "room": 1–32 символа [a-z0-9-], "cell": целое 0–8, не bool}`, других ключей нет. Ошибки по порядку: `"не объект"`; для `type`, `room`, `cell` — `"<ключ>: нет"` или `"<ключ>: неверно"`; затем `"лишний ключ: <k>"` по алфавиту.

```
Ввод:  validate_move({"type": "move", "room": "r-1", "cell": 4})
       validate_move({"type": "mv", "cell": True, "x": 1, "a": 2})
Вывод: []
       ["type: неверно", "room: нет", "cell: неверно",
        "лишний ключ: a", "лишний ключ: x"]
```

**A4*.** `seeding(n)` — посев олимпийской сетки на `n = 2^k` участников, чтобы сильнейшие встретились как можно позже: из посева на `n/2` каждое `s` превращается в пару `s, n + 1 − s`. Не степень двойки — `ValueError`. Почему 1 и 2 могут встретиться только в финале?

```
Ввод:  seeding(4), seeding(8)
Вывод: [1, 4, 2, 3], [1, 8, 4, 5, 2, 7, 3, 6]
```

## B. Протоколы

**B1.** `parse_request(raw)` — HTTP/1.1-запрос, строки через `\r\n`: `{"method", "path", "query" (последнее значение побеждает), "headers" (имена в нижнем регистре), "body"}`. Неверная первая строка — `ValueError`.

```
Ввод:  parse_request("GET /api/rooms?page=2&sort=new&page=3 HTTP/1.1\r\n"
                     "Host: game.example\r\n\r\n")
Вывод: {"method": "GET", "path": "/api/rooms", "query": {"page": "3", "sort": "new"},
        "headers": {"host": "game.example"}, "body": ""}
```

**B2.** `ws_header(data)` — заголовок кадра WebSocket (RFC 6455): FIN — старший бит байта 0, opcode — младшие 4 бита; MASK — старший бит байта 1, длина — 7 бит: меньше 126 — она сама, 126 — следующие 2 байта, 127 — 8 байт; с маской ещё 4 байта ключа. Ответ `{"fin", "opcode", "masked", "length", "header"}`. Не хватает байтов — `ValueError`.

```
Ввод:  ws_header(bytes([0x81, 0x05]))
       ws_header(bytes([0x81, 0xFE, 0x01, 0x00]) + b"key!")
Вывод: {"fin": True, "opcode": 1, "masked": False, "length": 5, "header": 2}
       {"fin": True, "opcode": 1, "masked": True, "length": 256, "header": 8}
```

**B3.** `verify_init_data(init_data, bot_token)` — проверка `initData` Mini App: разобрать как query-строку, убрать `hash`; строка проверки — `ключ=значение` по алфавиту через `\n`; `secret = HMAC-SHA256(ключ b"WebAppData", bot_token)`; `hash = hex HMAC-SHA256(secret, строка)`. Совпало (`hmac.compare_digest`) — поля без `hash`, иначе `None`.

```
Ввод:  s = sign_init_data({"auth_date": "1777777777", "query_id": "AAE1"}, TOKEN)
       verify_init_data(s, TOKEN), verify_init_data(s, TOKEN + "x")
Вывод: {"auth_date": "1777777777", "query_id": "AAE1"}, None
```

## C. Алгоритмы

**C1.** `swiss_pairs(points, played)` — тур швейцарки: игроки по убыванию очков, при равенстве — по имени; первый свободный играет с ближайшим ниже, с кем ещё не играл; если дальше пары не собрать — перебор с возвратом. Пар нет — `ValueError`.

```
Ввод:  swiss_pairs({"a": 2, "b": 1, "c": 1, "d": 0},
                   {frozenset("ab"), frozenset("ac"), frozenset("cd")})
Вывод: [("a", "d"), ("b", "c")]
```

**C2.** `latency(graph, src)` — кратчайшие задержки от `src` (Дейкстра, рёбра направленные, мс) до всех достижимых узлов, по алфавиту.

```
Ввод:  latency({"tg": {"nginx": 20}, "nginx": {"api1": 2, "api2": 5},
                "api1": {"redis": 1, "pg": 4}, "api2": {"pg": 1},
                "redis": {"pg": 1}}, "tg")
Вывод: {"api1": 22, "api2": 25, "nginx": 20, "pg": 24, "redis": 23, "tg": 0}
```

**C3.** `migration_order(revs)` — ревизии Alembic: ревизия → `down_revision` (`None` — первая). Порядок применения до head. Несколько корней или голов, ссылка на неизвестную ревизию, цикл — `ValueError`. Что делает `alembic merge`, когда у двух веток по голове?

```
Ввод:  migration_order({"c3": "b2", "a1": None, "b2": "a1", "d4": "c3"})
       migration_order({"a": None, "b": "a", "c": "a"})
Вывод: ["a1", "b2", "c3", "d4"]
       ValueError
```

**C4*.** `diff_lines(a, b)` — построчный diff по наибольшей общей подпоследовательности: `" x"` — общая, `"- x"` — удалена, `"+ x"` — добавлена; при выборе — сначала удаление.

```
Ввод:  diff_lines(["a", "b", "c", "d"], ["a", "c", "x", "d"]), diff_lines(["x"], ["y"])
Вывод: ["  a", "- b", "  c", "+ x", "  d"], ["- x", "+ y"]
```

## D. Релиз

**D1.** `semver_sort(versions)` — сортировка по semver: `1.0.0-rc1 < 1.0.0`; части пред-релиза через точку, числа меньше строк и сравниваются как числа. Можно с `v` впереди. Неверная версия — `ValueError`.

```
Ввод:  semver_sort(["1.0.0", "1.0.0-rc.10", "0.9.12", "1.0.0-rc.2", "1.0.1",
                    "1.0.0-beta"])
Вывод: ["0.9.12", "1.0.0-beta", "1.0.0-rc.2", "1.0.0-rc.10", "1.0.0", "1.0.1"]
```

**D2.** Conventional commits: `next_version(current, commits)` — `feat!:` или `BREAKING CHANGE` → мажор (при `0.y` — минор), `feat` → минор, `fix` → патч, иначе — `current`; пред-релиз отбрасывается. `changelog(version, date, commits)` — `## версия — дата` и непустые `### Ломающее`, `### Новое`, `### Исправлено`, пункты `- scope: описание`.

```
Ввод:  cm = ["feat(hints): подсказки", "fix: рейтинг при ничьей",
             "chore: обновить ruff"]
       next_version("1.0.0-rc1", cm), next_version("1.2.3", ["feat!: новый протокол"])
       print(changelog("1.1.0", "2027-05-20", cm))
Вывод: "1.1.0", "2.0.0"
       ## 1.1.0 — 2027-05-20
       ### Новое
       - hints: подсказки
       ### Исправлено
       - рейтинг при ничьей
```

**D3.** `scan_secrets(text)` — проверка перед публикацией репозитория: `(номер строки, вид)`. Виды: `"telegram"` — токен бота (8–10 цифр, `:`, 35 символов `[A-Za-z0-9_-]`), `"private_key"` — PEM-ключ, `"dsn_password"` — пароль в URL базы (`user:пароль@`, кроме `$VAR`, `${VAR}`, `<…>`). Что делать, если секрет уже в истории git?

```
Ввод:  scan_secrets("TOKEN=1234567890:" + "A" * 35 + "\n"
                    "DB=postgresql://game:hunter2@db/game\n"
                    "DB=postgresql://game:${PW}@db/game")
Вывод: [(1, "telegram"), (2, "dsn_password")]
```

## E. Проект и алгоритмы

**E1.** Сценарий финального демо (лекция 28) по секундам и запасной план: покажите видео-демо и второй телефон, проверенные на прогоне.

**E2.** Отчёт команды о работе с агентами за год `docs/agents-report.md`: два примера, где агент помог, и два, где ошибся, со ссылками на PR.

**E3.** Тег `v1.0.0-rc1` на staging, CHANGELOG и README с быстрым стартом, проверенным на чистой машине (семинар 27).

**E4 · босс.** Турнир финала. За каждую часть свой плюс.

а) `round_robin(players)` — круговой турнир «каруселью»: при нечётном числе добавить `"BYE"` (пары с ним не выводим). Тур — пары `(p[i], p[n-1-i])`; потом первый неподвижен, последний становится

вторым.

```
Ввод:  round_robin(["A", "B", "C", "D"])
Вывод: [[("A", "D"), ("B", "C")], [("A", "C"), ("D", "B")], [("A", "B"), ("C", "D")]]
```

б) `standings(results)` — `(a, b, очки a)`. Победа — 3, ничья — 1. По убыванию очков; при равенстве — по очкам в личных встречах только между равными, затем по имени.

```
Ввод:  R = [("A", "B", 1), ("C", "D", 0.5), ("A", "C", 0), ("B", "D", 1), ("A", "D", 1),
            ("B", "C", 1)]
       standings(R)
Вывод: [("A", 6), ("B", 6), ("C", 4), ("D", 1)]
```

в) `elo_replay(start, results, k=32)` — прогнать результаты по порядку через A1, новичок — 1200. Ответ — по алфавиту.

```
Ввод:  elo_replay({"A": 1300}, R)
Вывод: {"A": 1302, "B": 1222, "C": 1204, "D": 1172}
```

**E5*.** Подбор соперников. `window(wait) = 100 + 50 · ⌊wait / 10⌋` — допустимая разница рейтингов. `matchmake(events, horizon, tick=1)` — события `(время, игрок, рейтинг)`; каждый тик от 0 до `horizon` добавляем пришедших, перебираем ожидающих по времени прихода (затем по имени); свободному ищем свободного соперника с `|Δ| ≤ min(окон обоих)`, из подходящих — с наименьшей `|Δ|`, затем пришедшего раньше, затем по имени. Ответ — `(момент, первый, второй, |Δ|)`.

```
Ввод:  matchmake([(0, "ann", 1500), (0, "bob", 1720), (2, "cid", 1580),
                  (5, "dan", 1650), (12, "eve", 1500), (20, "fay", 1690)], 60)
Вывод: [(2.0, "ann", "cid", 80), (5.0, "bob", "dan", 70), (40.0, "eve", "fay", 190)]
```